

Class 13: RNASeq with DESeq2

Daniel Kim

Background

Today we will perform an RNASeq analysis of the effects of a common steroid on airway cells.

In particular, dexamethasone (hereafter just called “dex”) on different airway smooth muscle cell lines (ASM cells).

Data Import

We need two different inputs:

- **countData**: with genes in rows and experiments in columns
- **colData**: meta data that describes the columns in countData

```
counts <- read.csv("airway_scaledcounts.csv", row.names=1)
metadata <- read.csv("airway_metadata.csv")
```

```
head(counts)
```

	SRR1039508	SRR1039509	SRR1039512	SRR1039513	SRR1039516
ENSG000000000003	723	486	904	445	1170
ENSG000000000005	0	0	0	0	0
ENSG000000000419	467	523	616	371	582
ENSG000000000457	347	258	364	237	318
ENSG000000000460	96	81	73	66	118
ENSG000000000938	0	0	1	0	2
	SRR1039517	SRR1039520	SRR1039521		
ENSG000000000003	1097	806	604		
ENSG000000000005	0	0	0		
ENSG000000000419	781	417	509		
ENSG000000000457	447	330	324		

ENSG00000000460	94	102	74
ENSG00000000938	0	0	0

```
head(metadata)
```

	id	dex	celltype	geo_id
1	SRR1039508	control	N61311	GSM1275862
2	SRR1039509	treated	N61311	GSM1275863
3	SRR1039512	control	N052611	GSM1275866
4	SRR1039513	treated	N052611	GSM1275867
5	SRR1039516	control	N080611	GSM1275870
6	SRR1039517	treated	N080611	GSM1275871

Q1. How many genes are in this dataset?

```
dim(counts)
```

```
[1] 38694      8
```

38694 genes.

Q2. How many ‘control’ cell lines do we have?

```
table(metadata$dex)
```

control	treated
4	4

4 control cell lines.

Differential gene expression

We have 4 replicate drug treated and control (no drug) columns/experiments in our `counts` object.

We want one “mean” value for each gene (rows) in “treated” (drug) and one mean value for each gene in “control” cols.

Step 1. Find all “control” columns in `counts` Step 2. Extract these columns to a new object called `control.counts` Step 3. Then calculate the mean value for each gene

Q4. Follow the same procedure for the treated samples (i.e. calculate the mean per gene across drug treated samples and assign to a labeled vector called treated.mean)

Step 1

```
control.inds <- metadata$dex == "control"  
treated.inds <- metadata$dex == "treated"
```

Step 2

```
control.counts <- counts[, control.inds]  
treated.counts <- counts[, treated.inds]
```

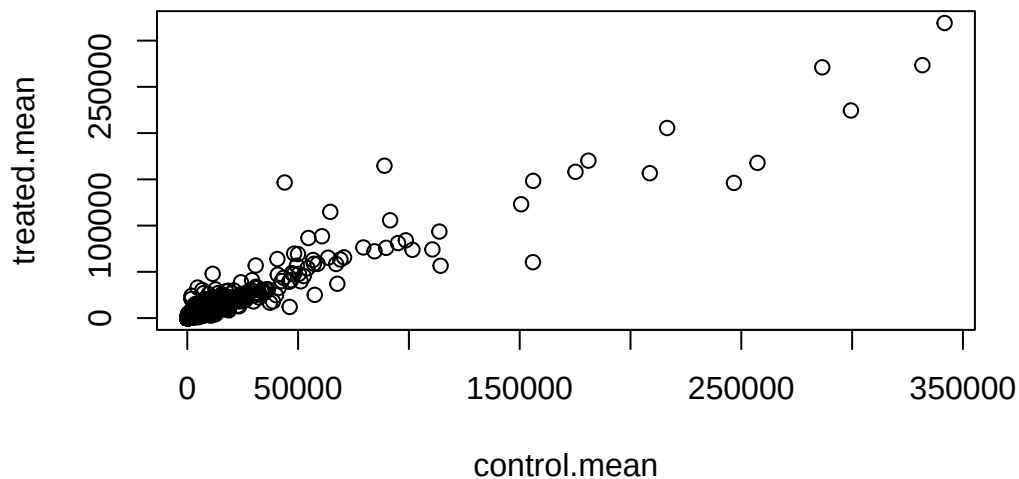
Step 3

```
control.mean <- rowMeans(control.counts)  
treated.mean <- rowMeans(treated.counts)
```

```
meancounts <- data.frame(control.mean, treated.mean)
```

Q5 (a). Create a scatter plot showing the mean of the treated samples against the mean of the control samples. Your plot should look something like the following.

```
plot(meancounts)
```



Q5 (b). You could also use the `ggplot2` package to make this figure producing the plot below. What `geom_?()` function would you use for this plot?

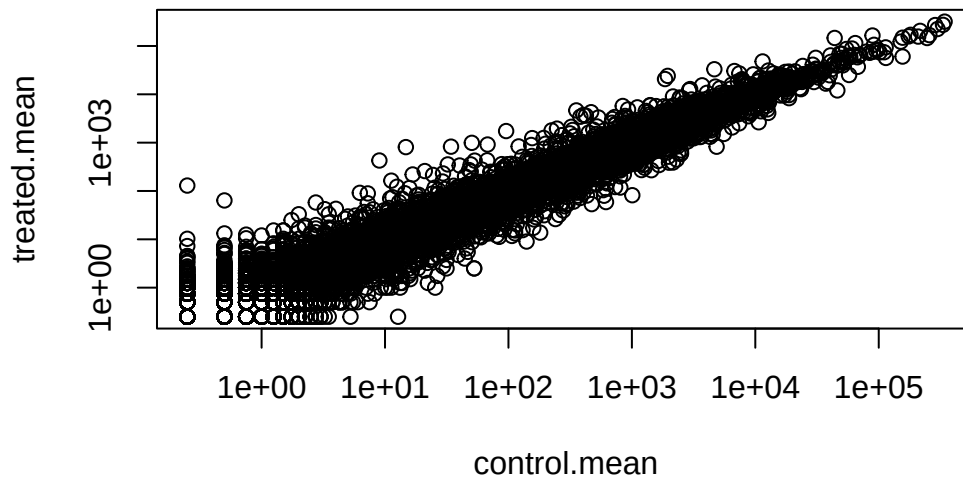
`geom_point()`.

Q6. Try plotting both axes on a log scale. What is the argument to `plot()` that allows you to do this?

```
plot(meancounts, log="xy")
```

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15032 x values ≤ 0 omitted from logarithmic plot

Warning in `xy.coords(x, y, xlabel, ylabel, log)`: 15281 y values ≤ 0 omitted from logarithmic plot



```
library(DESeq2)
```

DESeq (like many bioconductor packages) want its input data in a very specific way.

```
dds <- DESeqDataSetFromMatrix(countData = counts, colData = metadata, design = ~dex)
```

converting counts to integer mode

Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in design formula are characters, converting to factors

Run the DESeq analysis pipeline

```
dds <- DESeq(dds)
```

estimating size factors

estimating dispersions

gene-wise dispersion estimates

mean-dispersion relationship

final dispersion estimates

fitting model and testing

```
res <- results(dds)
```

```
res
```

log2 fold change (MLE): dex treated vs control

Wald test p-value: dex treated vs control

DataFrame with 38694 rows and 6 columns

	baseMean	log2FoldChange	lfcSE	stat	pvalue
	<numeric>	<numeric>	<numeric>	<numeric>	<numeric>
ENSG000000000003	747.1942	-0.350703	0.168242	-2.084514	0.0371134
ENSG000000000005	0.0000	NA	NA	NA	NA
ENSG0000000000419	520.1342	0.206107	0.101042	2.039828	0.0413675
ENSG0000000000457	322.6648	0.024527	0.145134	0.168996	0.8658000
ENSG0000000000460	87.6826	-0.147143	0.256995	-0.572550	0.5669497

...	...	...	...	...	...
ENSG00000283115	0.000000	NA	NA	NA	NA
ENSG00000283116	0.000000	NA	NA	NA	NA
ENSG00000283119	0.000000	NA	NA	NA	NA
ENSG00000283120	0.974916	-0.66825	1.69441	-0.394385	0.693297
ENSG00000283123	0.000000	NA	NA	NA	NA

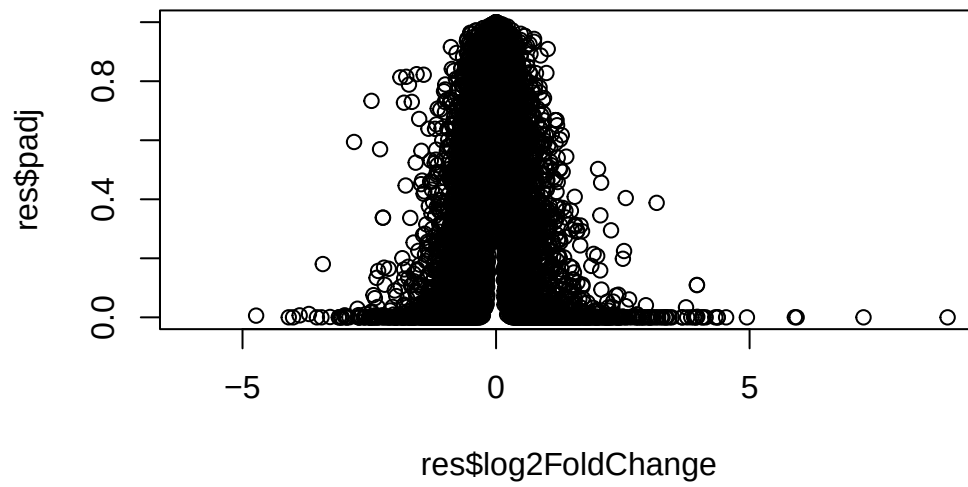
	padj
	<numeric>
ENSG000000000003	0.163017
ENSG000000000005	NA
ENSG0000000000419	0.175937
ENSG0000000000457	0.961682
ENSG0000000000460	0.815805

...	...
ENSG00000283115	NA
ENSG00000283116	NA
ENSG00000283119	NA
ENSG00000283120	NA
ENSG00000283123	NA

Volcano Plot

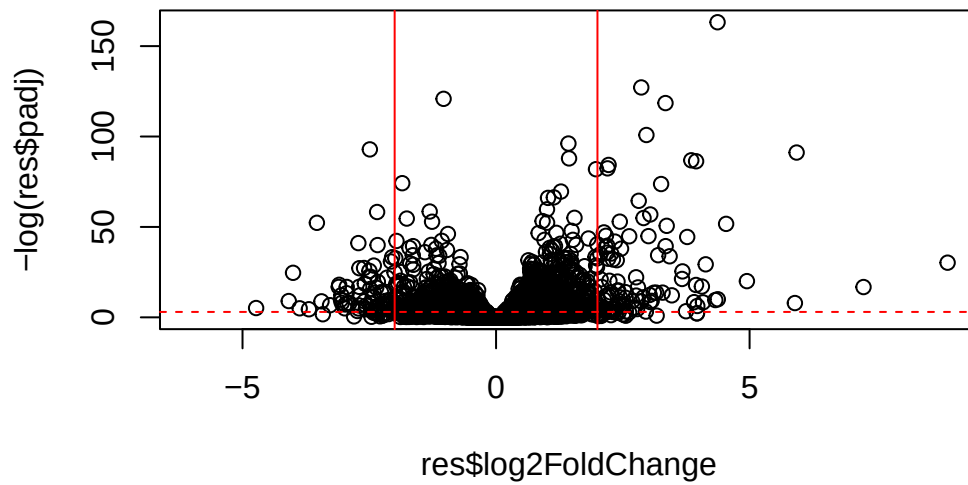
This is a main summary results figure from these kinds of studies. It is a plot of Log2-foldchange vs (Adjusted) P-value.

```
plot(res$log2FoldChange, res$padj)
```



Again this y-axis highly needs log transforming

```
plot(res$log2FoldChange, -log(res$padj))
abline(v=-2, col="red")
abline(v=2, col="red")
abline(h=-log(0.05), col="red", lty=2)
```

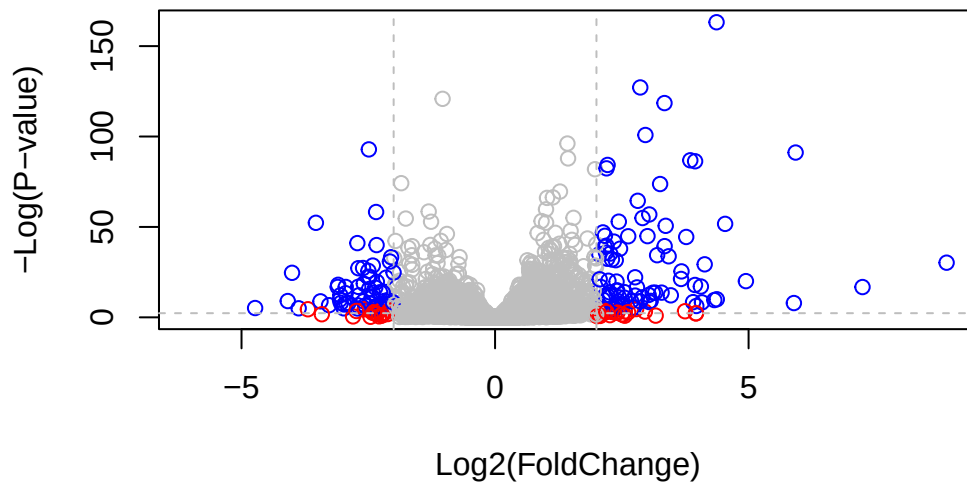


```
# Setup our custom point color vector
mycols <- rep("gray", nrow(res))
mycols[ abs(res$log2FoldChange) > 2 ] <- "red"

inds <- (res$padj < 0.01) & (abs(res$log2FoldChange) > 2 )
mycols[ inds ] <- "blue"

# Volcano plot with custom colors
plot( res$log2FoldChange, -log(res$padj),
      col=mycols, ylab="-Log(P-value)", xlab="Log2(FoldChange)" )

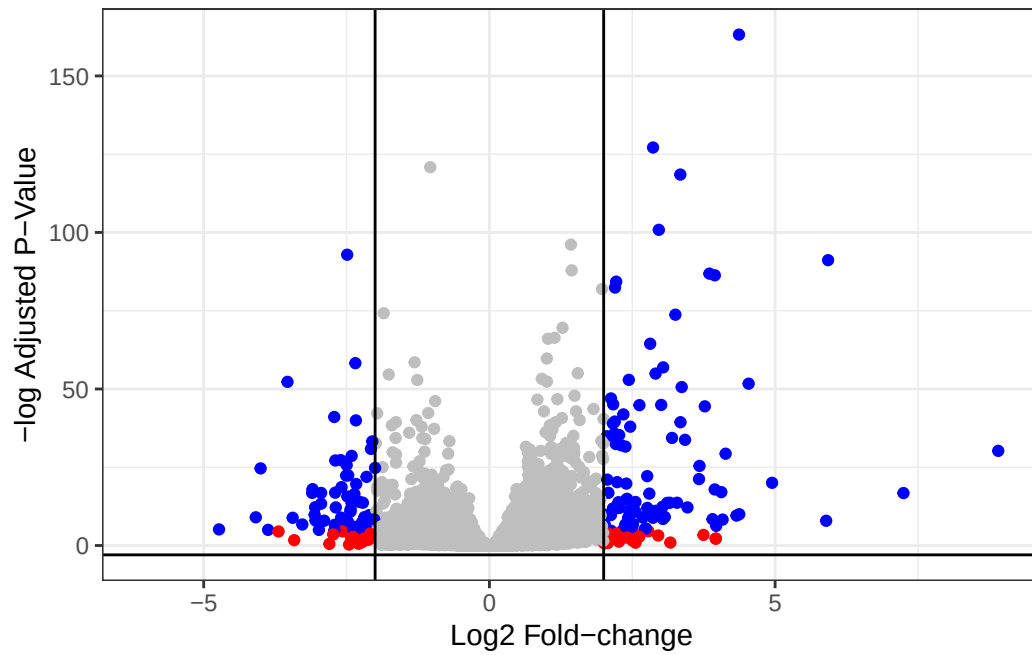
# Cut-off lines
abline(v=c(-2,2), col="gray", lty=2)
abline(h=-log(0.1), col="gray", lty=2)
```

Q. Make a presentation quality ggplot version of this plot. Include clear axis labels, a clean theme, your custom colors, cut-off lines and a plot title.

```
library(ggplot2)
ggplot(res) +
  aes(log2FoldChange, -log(padj)) +
  geom_point(color = mycols) +
  labs(x = "Log2 Fold-change", y = "-log Adjusted P-Value") +
  theme_bw() +
  geom_vline(xintercept=c(-2,2)) +
  geom_hline(yintercept=log(0.05))
```

Warning: Removed 23549 rows containing missing values or values outside the scale range (`geom\_point()`).



Save our results

Write a CSV file

```
write.csv(res, file="results.csv")
```